

第四讲 软件安全漏洞基础

一、软件漏洞简介

1、令人困惑的几个问题

(1) 我从不运行任何来历不明的软件，为什么还会中毒？

→ 如果病毒利用重量级的系统漏洞进行传播，您将在劫难逃。

因为系统漏洞可以引起计算机被远程控制，更何况传播病毒。横扫世界的冲击波蠕虫、slammer 蠕虫等就是这种类型的病毒。

→ 如果服务器软件存在安全漏洞，攻击者也可以发起“主动”进攻。

→ 这类病毒对我们影响很大的一个就应该是 LNK 快捷方式文件漏洞了。黑客可以通过精心构造的快捷方式文件来进行病毒传播。

→ 此漏洞存在于微软所有的操作系统中，包括 WIN7 和 Windows Server 2008，仅在中国受到波及的计算机就超过 2 亿台。

(2) 我只是点击了一个 URL 链接，并没有执行任何其他操作，为什么会中木马？

→ 如果浏览器在解析 HTML 文件时存在缓冲区溢出漏洞，那么攻击者就可以精心构造着承载着恶意代码的 HTML 文件，并且把链接发给你。当你点击这个链接时，漏洞就会被触发。而 HTML 中所承载的恶意代码 shellcode 被执行。

(3) Word 文档、PPT 文档、EXCEL 文档会导致恶意代码的执行么？

→ 和 html 文件一样，这类文档本身虽然是数据文件，但是如果 Office 软件在解析这些数据文件的特定数据结构时存在缓冲区溢出漏洞的话，攻击者就可以精心构造一个文档来出发并利用漏洞。

(4) 上网时，我使用高强度的密码注册账户，我的账户安全吗？

→ 高强度的密码只能抵抗密码暴力猜解的攻击，具体安全与否取决于很多其他因素：如密码存在哪里，密码怎样存，密码怎样传递等等。如果这些过程中有任何一处失误，都有可能引起密码泄漏。

→ 有些网站的账户信息比如密码是用明文存储的，这就无论我们存储什么样的密码都有可能被泄漏，最近的 CSDN 泄漏相信大家都有所耳闻，由于存储的是明文，那么即使你使用 ppnn13%dkstFeb.1st 这种密码也是没用的。

2、软件漏洞的概念

随着现代软件工业的不断发展，软件规模不断扩大，软件内部的逻辑也变得异常复杂。为了保证软件的质量，测试环节（QA）所投入的资源甚至已经超过了开发。即便如此，不论从理论上还是工程上都没有任何人敢声称能够彻底消灭软件中的所有逻辑缺陷--BUG

SQL 注入、跨站脚本、缓冲区溢出。

我们通常把这类能够引起软件做一些“超出设计范围的事情”的 bug 称为漏洞。

	含义	例子
BUG	功能性逻辑缺陷，影响软件的正常功能	执行结果错误、图标显示错误等
漏洞	安全性逻辑缺陷，通常情况下不影响软件的正常功能，但被攻击者成功利用后，有可能引起软件去执行额外的恶意代码。	软件缓冲区溢出漏洞、网站的跨站脚本漏洞(XSS)、SQL注入漏洞等

3、软件漏洞的分类

(1) 缓冲区溢出漏洞：它发生的原理是，由于用户处理用户数据时使用了不限边界的拷贝，导致程序内部一些关键数据被覆盖，引发了安全问题，严重的缓冲区溢出漏洞会使得程序被利用而安装上木马或病毒。

(2) 整数溢出漏洞：对于有符号的 16 位整数来说，它的最大值就是 0x7fff，也就是十进制的 32767，当赋值给一个整数的值为其最大值时，如果此时再加上 1，就会发生整数型的溢出。

```
int num;
num = argv[1];
if(num+1<50000)
{
    strcpy(des,src,num);
}
```

代码判断num的值，来判断是否执行strcpy函数。如果外部赋值为0x7fffffff，这样当程序执行到if语句时，num+1的值反而会变成负数，因而此时数值已经超过了32为有符号整数的最大值，变成负数，执行了字符串拷贝函数，会造成一个潜在的漏洞。

(3) 格式化字符串漏洞：格式化字符串漏洞是由于程序的数据输出函数中对输出数据的格式解析不当而发生的。

格式化字符	意义
%d	以十进制形式输出带符号整数（正数不输出符号）
%o	以八进制形式输出无符号整数（不输出前缀0）
%x	以十六进制形式输出无符号整数（不输出前缀0x）
%u	以十进制形式输出无符号整数
%f	以小数形式输出单、双精度实数
%e	以指数形式输出单、双精度实数
%g	以%f%e中较短的输出宽度输出单、双精度实数
%c	输出单个字符
%s	输出字符串

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    printf("%c\n","abcdef");//输出字符串应该为%s
```

```
    return 0;
```

```
}
```

当我们运行这个程序时，我们发现的输出居然是一个数字 4，而不是字符串 “abcdef”。

这是因为“%c”这个符号也是一个特殊符号，它代表的意思是输出一个字母。但是，我们提供给它的数据是一段字符串“abcdef”，这个时候程序就发生了解析错误。在一定条件下，恶意代码可以利用这种错误的解析来执行任意程序。

(4) SQL 注入漏洞：SQL 语句就会产生异常的结果，造成信息泄漏或者越权登录等情况。

这种通过控制传递给软件数据库操作语句的关键变量来获得恶意控制软件数据库、获取有用信息或者制造恶意破坏的，甚至是控制用户计算机系统的漏洞，就成为“SQL 注入漏洞”。

```
select * from user where username="" and password=""
```

引号中的就是帐号密码输入框传入的信息。

如果我们传入的信息为 ' or '1'='1',那么这个语句就会变成

```
select * from user where username="" or '1'='1' and password="" or '1'='1'
```

4、软件漏洞的危害

(1) 无法正常使用

对于软件安全漏洞来说，最直观的表现就是造成软件无法正常使用。

以 win98 为例，该操作系统曾经出现过再处理过多 ping 数据包时出现蓝屏的拒绝服务漏洞。

当我们用一台计算机连续不断地发送 ping 数据包到一台 Windows 98 系统的计算机时，该计算机就马上出现系统蓝屏死机的现象。

(2) 引发恶性事件

如果一个航天飞机上使用的导航控制软件出现了问题，在运行中错误地计算了飞机飞行的轨道，那么航天飞机上的宇航员就可能面临生命的危险。

计算机软件还可能被使用在注入国家安全局这样的情报部门，当一个软件发生信息泄露漏洞时，就会造成大量的国家绝密信息被泄露，一旦敌对国家或者组织获得这些信息，那就可能造成一个国家陷入到极其可怕的境地中。

(3) 关键数据丢失

前几年，某杀毒软件曾将系统关键文件当作病毒进行了彻底删除，造成了用户的计算机系统无法正常启动，很多数据信息因此丢失，虽然引发这场危机的罪魁祸首是软件的误删除错误，而不是一个安全漏洞，但是我们能够看到软件出现问题后所造成的影响。

一些软件出现漏洞后，可能被恶意攻击者利用而进行对用户关键数据的删除，从而造成用户数据的丢失。有一些漏洞甚至会直接造成对用户数据的清除。

(4) 秘密信息泄漏

对于信息泄漏漏洞来说，用户的计算机系统等于是赤裸裸地摆在了别人的面前，里面的信息都能被其他人随意获取、查看。

对于个人是这样，对于单位、政府部门会造成更大的影响。一些敏感的有关国家军事安全、企业核心经济利益的信息，都有可能被一个软件的安全漏洞所泄漏，造成无法预知的后果。

(5) 被安装木马病毒

一个软件漏洞被发现后，最常见的利用就是恶意攻击者借此来散布木马病毒程序。我们的计算机系统中也许安装有最新的杀毒软件，最好的防火墙，但是只要计算机系统上的软件存在漏洞，那么恶意攻击者就能够在你的计算机系统中安装木马病毒程序。

5、软件漏洞出现的原因

(1) 小作坊式的软件开发

严格地讲，任何一款计算机软件都必须依据软件工程的思想来进行设计开发。

但是，出于种种原因，很多软件的开发并没有按照软件工程的要求来实现。

有些小型公司采用了直接开发，或者边设计边开发的方法，这样只做出来的软件犹如小作坊里生产出来的产品，难免存在很多漏洞。

(2) 赶进度带来的弊端

很多大型的软件公司即使采用了软件工程思想来设计软件，但是由于事件紧迫，任务繁重，也会在一定程度上采用投机取巧或者省工省料的办法来开发软件。这个时候开发出来的软件，往往由于开

发者过于疲劳或者赶进度，从而将不安全的因素带进软件，造成软件存在漏洞。

(3) 被轻视的安全测试

软件安全测试不但可以进行对软件代码的安全测试，还可以对软件成品进行安全测试。但是这样会增加成本，于是一些开发商要么不做，要么仅做最低级的测试。

这样只能保证软件能够正常使用，基本功能实现，其实，漏洞就这样被隐藏在软件内部。

(4) 淡薄的安全思想

软件安全思想主要是针对软件开发中最辛苦的编程人员来说的。由于编程人员对软件安全的认识并不一样，在做产品开发时，他们编写的代码也就对软件的安全出现了不同程度的影响。如果一个编程人员不具有一些基本的安全编程经验，他可能就会把最简单最常见的安全漏洞引入到软件内部。

(5) 不完善的安全维护

当软件出现安全漏洞时，这并不可怕。软件的开发商只需要认真检查软件出现漏洞的原因，找出修补方案就可以弥补漏洞带来的危害与损失。

可是有些开发商知道软件出现漏洞也不修补甚至欺骗用户。这种情况下，软件就会一直存在漏洞，那么使用这样软件的用户就会面临着危险。

二、漏洞挖掘、分析、利用简介

1、漏洞挖掘简介

安全性漏洞往往不会对软件的本身功能造成很大影响，因此很难被 QA 工程师的功能性测试发现，对于“正常操作”的普通用户来说，更难体会到软件中的这类逻辑瑕疵了。

(1) 由于安全性漏洞往往有极高的利用价值。

例如，导致计算机被非法远程控制，数据库数据泄漏等，所以总是有无数技术精湛、精力旺盛的家伙在夜以继日地寻找软件中的这类逻辑瑕疵。

他们精通二进制、汇编语言、操作系统底层的知识也是杰出的程序员，因此能够敏锐的捕捉到程序员所犯的细小错误。

(2) 寻找漏洞的人并非全是攻击者。

大型的软件企业也会雇佣一些安全专家来测试自己产品中的漏洞，这种测试工作被称作为 Penetration test(攻击测试)，这些测试团队则被称作 Tiger team 或者 Ethic hacker。

(3) 从技术角度讲，漏洞挖掘实际上是一种高级的测试(QA)。

学术界一直热衷于使用静态分析的方法寻找源代码中的漏洞；而在工程界，不管是安全专家还是攻击者，普遍采用的漏洞挖掘方法是 Fuzz，这实际是一种“灰”盒测试。

2、漏洞分析简介

当 fuzz 捕捉到软件一个严重的异常时，当你想透过厂商公布的简单描述了解漏洞细节的时候，我们就需要具备一定的漏洞分析能力。一般情况下，我们需要调试二进制级别的程序。

在分析漏洞时，如果能够搜索到 POC(proof of concept 为观点提供证据)代码就能重现漏洞被出发的现场。这时可以使用调试器观察漏洞的细节，或者利用一些工具更方便的找到漏洞的触发点。

当无法获得 POC 时，就只有厂商提供的对漏洞的简单描述了。一个比较通用的办法是使用补丁比较器，首先比较 patch 前后的可执行文件有哪些地方被修改，之后可以利用反汇编工具来重点逆向分析这些地方。

漏洞分析需要扎实的逆向基础和调试技术，除此之外还要精通各种场景下的漏洞利用方法。这种技术更多依靠的是经验，很难总结出通用的条款。

3、漏洞利用简介

漏洞利用技术可以一直追溯到 20 世纪 80 年代的缓冲区溢出漏洞的利用。

随着时间的推移，经过无数安全专家和黑客们针锋相对的研究，这项技术已经在多种流行的操作系统和编译环境下得到了实践，并日趋完善。这包括内存漏洞(堆栈溢出)和 Web 应用漏洞(脚本注入)等。

4、漏洞的公布与 Oday 响应

(1) 漏洞公布的流程取决于漏洞是被谁发现的。

如果是安全专家、Pen Test、Ethical Hacker 在测试中发现了漏洞，一般会立刻通知厂商的产品安全中心。软件厂商在经过漏洞确认、补丁测试之后，会正式发布漏洞公告和官方补丁。

如果漏洞被攻击者找到，肯定不会立刻通知软件厂商。这是漏洞的信息只有攻击者自己知道，他可以写出 exploit 利用漏洞来做任何事情。这种未被公布、未被修复的漏洞往往被称做 0day。

0day 漏洞是危害最大的漏洞，当然对攻击者来说也是最有价值的漏洞。

(2) 公布漏洞的权威机构有两个。

→ CVE(Common Vulnerabilities and Exposures)

<http://cve.mitre.org> 截至目前，这里收录了两万多个漏洞。CVE 会对没个公布的漏洞进行编号、审查。CVE 编号通常也是引用漏洞的标准方式。

→ CERT(Computer Emergency Response Team)

<http://www.cert.org> 计算机应急相应组往往会第一时间跟进当前的严重漏洞，包括描述信息、POC 的发布链接、厂商的安全相应进度、用户应该才去的临时性防范措施等。

(3) 1day 攻击

微软的安全中心所公布的漏洞也是所有安全工作者和黑客们最感兴趣的地方。微软每个月第二周的星期二发布补丁，这一天通常成为“Black Tuesday”，因为会有许多攻击者通宵达旦地研究这些补丁 patch 了哪些漏洞，并写出 exploit。因为在补丁刚刚发布的一段时间内，并非所有用户都能及时修复，故这种新公布的漏洞也有一定利用价值。

有时候把攻击这种刚刚 patch 过的漏洞成为 1day 攻击。

三、PE 文件格式简介

PE(Portable Executable)是 Win32 平台下可执行文件遵守的数据格式。常见的可执行文件(如 “*.exe” 文件和 “*.dll” 文件)都是典型的 PE 文件。

一个可执行文件不光包含了二进制的机器代码，还会包含许多其他信息，如字符串、菜单、图标、位图、字体等。

PE 文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照 PE 文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。

PE 文件格式把可执行文件分成若干个数据节(section)，不同的资源被存放在不同的节中。一个典型的 PE 文件包含的节如下：



● DOS 头是用来兼容 MS-DOS 操作系统的，目的是当这个文件在 MS-DOS 上运行时提示一段文字，大部分情况下：This program cannot be run in DOS mode.还有一个目的，就是指明 NT 头在文件中的位置。

● NT 头包含 windows PE 文件的主要信息，其中包括一个 ‘PE’ 字样的签名，PE 文件头 (IMAGE_FILE_HEADER) 和 PE 可选头 (IMAGE_OPTIONAL_HEADER32)，头部的详细结构以及其具体意义在 PE 文件头文章中详细描述。

● 节表：是 PE 文件后续节的描述，windows 根据节表的描述加载每个节。

● 节：每个节实际上是一个容器，可以包含代码、数据等等，每个节可以有独立的内存权限，比如代码节默认有读/执行权限，节的名称和数量可以自己定义，必是上图中的三个。

其他	→ .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。
.data 节	→ .data 初始化的数据块，如宏定义、全局变量、静态变量等。
.rdata 节	→ .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。
.text 节	→ .rsrc 存放程序的资源，如图标、菜单等。
文件头、节表等	

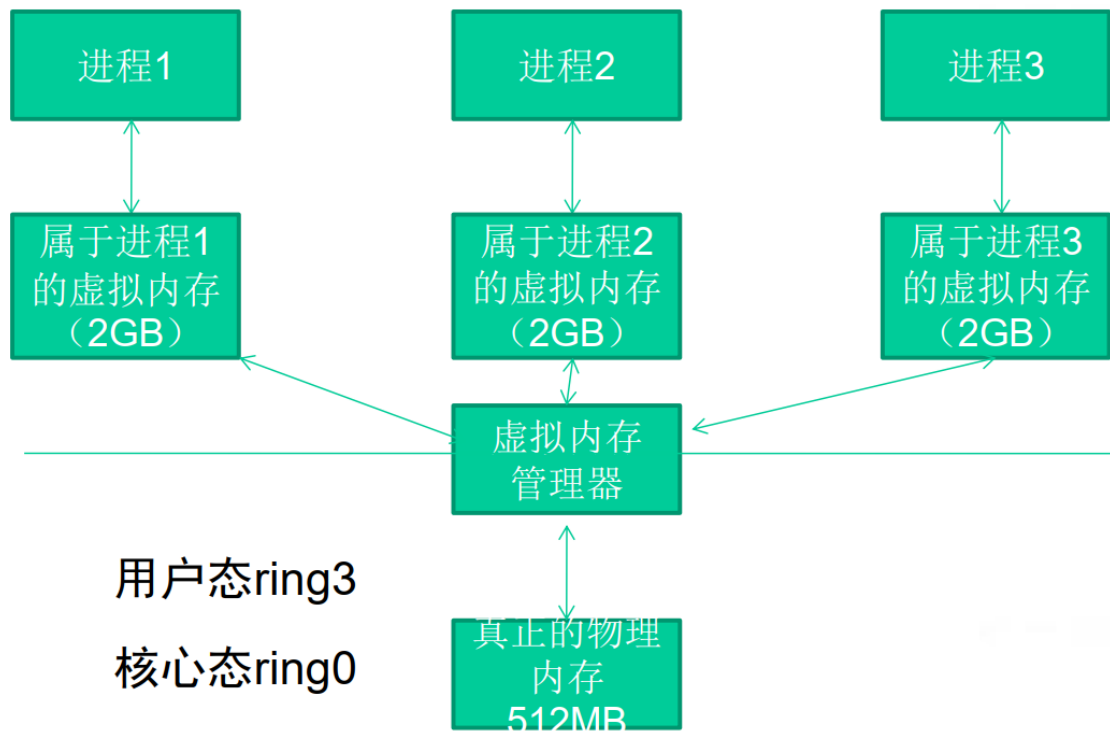
→ 除此之外，还可能出现的节包括 “.reloc”、“.edata”、“.tls”、“.rdata” 等。

四、虚拟内存相关知识

1、虚拟内存简介

Windows 的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入 Windows 内核级别 ring0 才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。

Windows 让所有的进程都“相信”自己拥有独立的 4GB 内存空间。但是我们计算机中那跟实际的内存条可能只有 512MB,怎么能为所有进程都分配 2GB 的内存呢？这一切都是通过虚拟内存管理器的映射做到的。



● 虽然每个进程都“相信”自己拥有 2GB 的空间，但实际上它们运行时真正能用到的空间根本没有那么多。

● 内存管理器只是分给进程一片“假地址”，或者说是“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；

● 当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。

2、内存管理与银行的类比

内存管理	银行类比
进程	储户
内存管理器	银行
物理内存	钞票
虚拟内存	存款
进程可能拥有大片内存，但使用往往很少	储户拥有大笔存款，但实际生活中的开销并没多少

内存管理	银行类比
进程不使用虚拟内存时，这些内存只是些地址，是虚拟存在的，是一笔无形的数字财富。	用户不使用储蓄时，储蓄也只是一些数字，是无形的数字财富。
进程使用内存时，内存管理器会为器会为这个虚拟地址映射实际的物理地址，虚拟内存地址和最终被映射到的物理内存地址之间没有什么必然联系	储户需要钱时，银行才会兑换一定的现金给储户，但物理钞票的号码与储户心目中的数字存款之间可能没有任何联系。

内存管理	银行类比
操作系统的实际物理内存空间可以远远小于进程的虚拟内存空间之和，仍能正常调度	银行的现金准备可以远远小于所有储户的储蓄额总和，仍能正常运转
很少出现所有程序要申请出全部物理内存	很少会出现所有储户同时要取出全部存款的现象
物理内存可以远小于虚拟内存之和	社会上实际流通的钞票可以远远小于社会的财富总额

3、PE 文件与虚拟内存之间的映射

● 静态反汇编工具看到的 PE 文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置

● 反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到 PE 文件中找到这条指令对应的机器码

(1) 文件偏移地址(File Offset)

➤ 数据在 PE 文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

(2) 装载基址(Image Base)

➤ PE 装入内存时的基地址。默认情况下，EXE 文件在内存中的基地址是 0x00400000，DLL 文件是 0x10000000。这些位置可以通过修改编译选项更改。

(3) 虚拟内存地址(Virtual Address,VA)

➤ PE 文件中的指令被装入内存后的地址。

(4) 相对虚拟地址(Relative Virtual Address,RVA)

➤ 相对虚拟地址是内存地址相对于映射基址的偏移量。

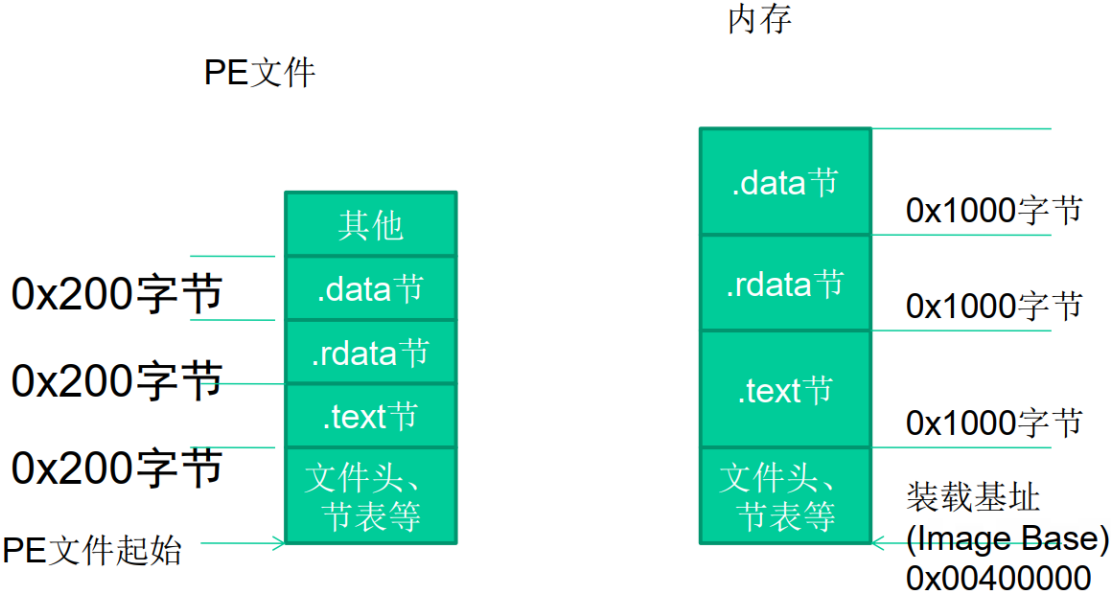
➔ 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系 $VA = Image\ Base + RVA$

(5) 文件偏移地址在与他们计算时还需要考虑存放方式的不同。

PE 文件中的数据按照磁盘数据标准存放，以 0x200 字节为基本单位进行组织。当一个数据节不足 0x200 字节时，不足的地方将被

0x00 填充；当一个数据节超过 0x200 字节时，下一个 0x200 块将分配给这个节使用。因此 PE 数据节的大小永远是 0x200 的整数倍。

当代码装入内存后，将按照内存数据标准存放，并以 0x1000 字节为基本单位进行组织。类似的，不足将被不全，若超出将分配下一个 0x1000 为其所用。因此，内存中的节总是 0x1000 的整数倍。



(6) 举例

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

我们把这种由存储单位差异引起的节基址差称做节偏移，在上图例中：

$$.text \text{ 节偏移} = 0x1000 - 0x400 = 0xc00$$

$$.rdata \text{ 节偏移} = 0x7000 - 0x6200 = 0xE00$$

$$.data \text{ 节偏移} = 0x9000 - 0x7400 = 0x1c00$$

$$.rsrc \text{ 节偏移} = 0x2D000 - 0x7800 = 0x25800$$

文件偏移地址

$$= \text{虚拟内存地址(VA)} - \text{装载基址(Image Base)}$$

$$- \text{节偏移} = \text{RVA} - \text{节偏移}$$

以上表为例，如果在调试时遇到虚拟内存中 0x00404141 处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

$$\begin{aligned} \text{文件偏移量} &= 0x00404141 - 0x00400000 - (0x1000 - 0x400) \\ &= 0x3541 \end{aligned}$$